Presentation Walkthrough - Modern Buffer Overflow Lab

This file is a step-by-step guide for presenting the project live.

Recommended presentation time:

```
Short version: 20–25 minutes
Full version: 35–45 minutes
```

Use the short version if you only need to demonstrate the main idea. Use the full version if you want to explain every phase of the penetration testing workflow.

0. Pre-Presentation Checklist

Run these before the presentation:

```
cd college-pentest-lab
docker compose down -v
docker compose up --build -d
docker compose ps
```

Expected:

```
attacker      Up
victim-l1     healthy
victim-l2     healthy
victim-l3     healthy
safe-victim   healthy
```

Enter the attacker container:

```
docker exec -it attacker bash
```

Keep another host terminal open for logs:

```
docker logs victim-l1 2>&1 | tail -30
```

1. Opening Slide - Project Idea

What to say

```
This project is a controlled penetration testing lab for learning stack-based
buffer overflow exploitation. It uses Docker containers so everything is local,
isolated, and safe for students.

The lab teaches the full flow:
Recon, fuzzing, crash analysis, offset discovery, exploitation, restricted
shell, post-exploitation basics, and mitigation.
```

Show this diagram

```
attacker container
    |
    | sends TCP commands
    v
victim-l1   basic overflow
victim-l2   NX enabled
victim-l3   canary + NX + PIE
safe-victim defensive version
```

Key point

```
The shell is restricted. It is designed for education, not for real system
abuse.
```

2. Architecture Slide

What to say

```
The attacker container contains the tools and scripts. Each victim container
runs a C TCP service. All vulnerable levels share the same source code, but
they are compiled with different protections.
```

Command

```
docker compose ps
```

Explain the ports

```
victim-l1    host port 9999
victim-l2    host port 9998
victim-l3    host port 9997
safe-victim  host port 9996
```

Inside Docker, the attacker talks to:

```
victim-l1:9999
victim-l2:9999
victim-l3:9999
safe-victim:9999
```

3. Tooling Slide

What to say

```
The attacker container includes the tools normally used in binary exploitation:
pwntools, checksec, gdb, pwndbg, ROPgadget, ropper, netcat, and Python.
```

Command

```
ls -la /targets
```

Expected files

```
level1
level2
level3
safe_server
libc.so.6
```

Key point

```
The binaries are copied automatically from the victim containers into
/targets, so students can analyze exactly what is running.
```

4. Recon Phase

What to say

```
First, we perform recon. In a real assessment, recon means identifying what is
running and what protections exist. Here the recon script connects to each
service and runs checksec on each binary.
```

Command

```
python3 scripts/00_recon.py
```

Explain the output

```
Level 1:
No canary, no PIE, stack executable. This is the easiest target.

Level 2:
NX is enabled. The stack is not executable, so stack shellcode is blocked.

Level 3:
Canary, NX, and PIE are enabled. Now we need a leak before exploitation.

Safe:
The code and compiler protections are defensive.
```

Transition

```
After recon, we know which target is easiest. We start with Level 1.
```

5. Fuzzing Phase

What to say

```
Fuzzing means sending different inputs to see if the program crashes. We do not
start by writing an exploit. We first prove that a crash exists.
```

Command

```
python3 scripts/01_fuzzer.py
```

Expected result

```
TRUN with 50 bytes  -> OK
TRUN with 100 bytes -> OK
TRUN with 150 bytes -> CRASH
```

Explain

```
The stack buffer is 128 bytes. Around 150 bytes, the input becomes large enough
to corrupt control data on the stack.
```

Optional host terminal command

```
docker logs victim-l1 2>&1 | tail -30
```

What to point out

```
RIP
RSP
RBP
Stack near RSP
```

6. Offset Discovery Phase

What to say

```
A crash is not enough. We need to know the exact number of bytes required to
reach the return address. This is called the offset.
```

Command

```
python3 scripts/02_find_offset.py
```

Explain the important number

```
Level 1 and Level 2:
128 bytes buffer + 8 bytes saved RBP = 136 bytes to return address.
```

Draw this

```
buffer[128]      AAAAAAAAA...
saved RBP[8]     AAAAAAAA
return address   controlled value
```

Key sentence

```
If we control the return address, we can redirect the program to code we choose.
```

7. Level 1 Exploit - Basic ret2win

What to say

```
Level 1 is the clean beginner exploit. There is a function called win() in the
binary. The exploit overwrites the return address with the address of win().
```

Command

```
python3 scripts/03_exploit_level1.py
```

Expected output

```
RESULT: SUCCESS - win() reached and restricted shell opened.
Flag: FLAG{stack_overflow_leads_to_code_execution}
```

Explain the payload

```
"A" * 136 + p64(win)
```

Explain the shell

```
After win() runs in Level 1, the lab opens a real /bin/sh inside the Docker
victim. This represents post-exploitation access, but the scope is still only
the isolated lab container.
```

Demo commands shown by the script

```
whoami
pwd
ls
cat flag.txt
id
exit
```

Optional interactive demo

```
python3 scripts/03_exploit_level1.py --interactive
```

Main lesson

```
Level 1 proves the most important concept: overwriting the return address gives
control over program execution.
```

8. Level 2 Exploit - NX Bypass

What to say

```
Level 2 adds NX. NX means the stack is not executable. So if an attacker puts
shellcode on the stack, the CPU should block it.

Instead of running new code from the stack, the exploit reuses existing code in
the binary. This is the beginner version of the same idea behind return-oriented
programming.
```

Command

```
python3 scripts/04_exploit_level2.py
```

Expected output

```
RESULT: SUCCESS - NX was bypassed by returning to existing code.
Restricted training shell opened.
```

Explain the payload

```
"A" * 136 + p64(win)
```

Key difference from Level 1

```
Level 1:
The target has almost no protections.

Level 2:
NX is enabled, so the lesson changes. We do not execute stack shellcode. We
return to code that already exists. Level 2 uses a restricted training shell
so the demo stays focused on the NX bypass concept.
```

Main lesson

```
NX does not remove the overflow bug. It changes the exploitation technique.
```

9. Level 3 Exploit - Leak First

What to say

```
Level 3 adds modern protections: stack canary, NX, and PIE.

The canary detects overwrites before the return address. PIE randomizes binary
addresses, so win() does not have a stable address.

To exploit this level, we need an information leak first.
```

Command

```
python3 scripts/05_exploit_level3.py
```

Explain the leak

The script sends:

```
INFO %1$p.%2$p
```

The vulnerable service leaks:

```
%1$p -> stack canary
%2$p -> runtime win() address after PIE relocation
```

Explain the payload

```
"A" * 136 + leaked_canary + "B" * 8 + leaked_win
```

Explain why this works

```
The payload preserves the correct canary, so the canary check passes.
The payload uses the leaked win() address, so PIE is bypassed.
Then the return address sends execution to win().
```

Expected output

```
RESULT: SUCCESS - canary and PIE were bypassed with the leak.
Restricted training shell opened.
```

Main lesson

```
Modern exploitation often needs two bugs or two phases:
first leak information, then use that information to exploit memory corruption.
```

10. Post-Exploitation Basics

What to say

```
After exploitation, the attacker usually wants to understand where they are,
what user they have, what files are available, and what proof they can collect.
In this lab, Level 1 demonstrates that with a real Docker-contained /bin/sh.
Level 2 and Level 3 use a restricted shell so students focus on the protections
being bypassed.
```

Useful shell commands

```
help
whoami
id
pwd
ls
cat flag.txt
hostname
uname -a
exit
```

Explain why it is restricted

```
This is an education lab. The shell is restricted so students learn the concept
without getting an unrestricted system shell.
```

11. Defense Demo - Safe Server

What to say

```
Now we compare the vulnerable service with a safe version. The safe server has
the same idea, but the dangerous behavior is removed.
```

Command

```
python3 scripts/06_verify_safe.py
```

Expected output

```
Oversized input rejected.
Format string treated as text.
Server remains alive.
```

Explain the fixes

```
1. Validate input length before copying.
2. Use bounded copy logic.
3. Use safe format strings.
4. Compile with hardening protections.
```

Main lesson

```
The best exploit is the one that never becomes possible because the code is
written and compiled safely.
```

12. Code Explanation Section

Use this if the audience asks how the code works.

Vulnerable copy

```
char buffer[128];
memcpy(buffer, input, input_len);
```

Explain:

```
The destination is fixed-size, but the copy length is attacker-controlled.
That is the root cause.
```

Safe copy

```
if (input_len >= SAFE_BUFFER_SIZE) {
    reject oversized input;
    return;
}
```

Explain:

```
The safe server rejects dangerous input before copying.
```

Safe format string

```
snprintf(response, sizeof(response), "%s\n", input);
```

Explain:

```
The user input is treated as data, not as a format string.
```

13. Suggested Slide Order

```
1. Title: Modern Buffer Overflow Lab
2. Problem: Why memory safety matters
3. Architecture: attacker, victims, safe server
4. Tools: Docker, pwntools, checksec, gdb
5. Recon: protection comparison
6. Fuzzing: finding the crash
7. Offset: controlling the return address
8. Level 1: ret2win
9. Level 2: NX bypass
10. Level 3: leak canary + PIE
11. Shell access: post-exploitation basics
12. Safe server: mitigation
13. Lessons learned
14. Q&A
```

14. Full Live Demo Commands

Run from host:

```
cd college-pentest-lab
docker compose up --build -d
docker compose ps
docker exec -it attacker bash
```

Run inside attacker:

```
python3 scripts/00_recon.py
python3 scripts/01_fuzzer.py
python3 scripts/02_find_offset.py
python3 scripts/03_exploit_level1.py
python3 scripts/04_exploit_level2.py
python3 scripts/05_exploit_level3.py
python3 scripts/06_verify_safe.py
```

Optional interactive sessions:

```
python3 scripts/03_exploit_level1.py --interactive
python3 scripts/04_exploit_level2.py --interactive
python3 scripts/05_exploit_level3.py --interactive
```

15. Short Demo Version

Use this if time is limited:

```
python3 scripts/00_recon.py
python3 scripts/03_exploit_level1.py
python3 scripts/04_exploit_level2.py
python3 scripts/05_exploit_level3.py
python3 scripts/06_verify_safe.py
```

Explain:

```
Recon shows protections.
Level 1 proves control-flow hijack.
```

```
Level 2 shows NX bypass by reusing existing code.
Level 3 shows leak-first exploitation.
Safe server proves mitigation.
```

16. Questions You May Be Asked

Is this a real shell?

Answer:

```
Level 1 opens a real /bin/sh, but only inside the victim Docker container.
Level 2 and Level 3 use a restricted training shell for safer, focused demos.
```

Why do Level 2 and Level 3 use restricted shells?

Answer:

```
The impact is kept understandable so students can focus on what changed in the
exploitation path: NX bypass and leak-before-exploit.
```

Why does Level 3 need a leak?

Answer:

```
Because the canary must be preserved and PIE randomizes code addresses. Without
the leak, the exploit does not know the correct canary or win() address.
```

Why not use a real reverse shell?

Answer:

```
For a college lab, the goal is learning safely. A restricted shell teaches the
same workflow without creating a general-purpose attack tool.
```

What is the main mitigation?

Answer:

```
Validate input length, use bounded copy logic, never use user input as a format
string, and compile with modern hardening protections.
```

17. Closing Summary

Use this as your final spoken summary:

```
This lab demonstrates how a simple memory-safety bug can become control over
program execution. Level 1 teaches the basic return-address overwrite. Level 2
shows that NX changes the technique but does not fix the bug. Level 3 shows why
modern exploitation often needs an information leak. Finally, the safe server
shows how secure coding and compiler protections break the attack chain.
```

18. Final Verification Checklist

Before submitting or presenting, verify:

```
docker compose ps
python3 scripts/00_recon.py
python3 scripts/03_exploit_level1.py
python3 scripts/04_exploit_level2.py
python3 scripts/05_exploit_level3.py
python3 scripts/06_verify_safe.py
```

Expected:

```
All services healthy.
All three exploit scripts open the restricted shell.
Safe server blocks the attack.
```